08. 피어 관리 및 디스커버리 구현 분석

개요

Lighthouse의 피어 관리 및 디스커버리 시스템은 Ethereum 2.0 네트워크에서 안정적이고 효율적인 P2P 통신을 위한 핵심 구성 요소입니다. 이 시스템은 피어의 연결 상태, 평판 관리, 자동 피어 발견, 그리고 네트워크 토폴로지 최적화를 담당합니다.

주요 구성 요소:

- **PeerManager**: 피어의 연결 상태와 평판을 관리하는 중앙 관리자
- **Discovery**: discv5 프로토콜을 기반으로 한 피어 발견 서비스
- **PeerDB**: 피어 정보와 메타데이터를 저장하는 데이터베이스
- **Score System**: 피어의 행동을 평가하고 관리하는 평판 시스템

PeerManager 핵심 구현

PeerManager 구조체

```
// beacon_node/lighthouse_network/src/peer_manager/mod.rs

pub  struct PeerManager<E: EthSpec> {

/// Storage of network globals to access the `PeerDB`.

network_globals: Arc<NetworkGlobals<E>>,

/// A queue of events that the `PeerManager` is waiting to produce.

events: SmallVec<[PeerManagerEvent; 16]>,

/// A collection of inbound-connected peers awaiting to be Ping'd.
```

```
inbound_ping_peers: HashSetDelay<PeerId>,

/// A collection of outbound-connected peers awaiting to be Ping'd.

outbound_ping_peers: HashSetDelay<PeerId>,

/// A collection of peers awaiting to be Status'd.

status_peers: HashSetDelay<PeerId>,

/// The target number of peers we would like to connect to.

target_peers: usize,

/// Peers queued to be dialed.

peers_to_dial: Vec<Enr>,

/// The number of temporarily banned peers.

temporary_banned_peers: LRUTimeCache<PeerId>,

/// A collection of sync committee subnets that we need to stay subscribed to.

sync_committee_subnets: HashMap<SyncSubnetId, Instant>,

/// A mapping of all custody groups to column subnets to avoid re-computation.

subnets_by_custody_group: HashMap<u64, Vec<DataColumnSubnetId>>,

/// The heartbeat interval to perform routine maintenance.

heartbeat: tokio::time::Interval,

/// Keeps track of whether the discovery service is enabled or not.

discovery_enabled: bool,

/// Keeps track of whether the QUIC protocol is enabled or not.

quic_enabled: bool,

trusted_peers: HashSet<Enr>,
```

```
}
```

PeerManager 이벤트 시스템

PeerManager는 이벤트 기반 아키텍처를 사용하여 네트워크 레이어와 통신합니다:

```
/// The events that the `PeerManager` outputs (requests).
#[derive(Debug)]
pub  enum PeerManagerEvent {
/// A peer has dialed us.
PeerConnectedIncoming(PeerId),
/// A peer has been dialed.
PeerConnectedOutgoing(PeerId),
/// A peer has disconnected.
PeerDisconnected(PeerId),
/// Sends a STATUS to a peer.
Status(PeerId),
/// Sends a PING to a peer.
Ping(PeerId),
/// Request METADATA from a peer.
MetaData(PeerId),
/// The peer should be disconnected.
DisconnectPeer(PeerId, GoodbyeReason),
/// Inform the behaviour to ban this peer and associated ip addresses.
```

```
Banned(PeerId, Vec<IpAddr>),

/// The peer should be unbanned with the associated ip addresses.

UnBanned(PeerId, Vec<IpAddr>),

/// Request the behaviour to discover more peers and the amount of peers to disc

DiscoverPeers(usize),

/// Request the behaviour to discover peers on subnets.

DiscoverSubnetPeers(Vec<SubnetDiscovery>),

}
```

피어 연결 관리

피어 연결 상태 추적

PeerManager는 각 피어의 연결 상태를 세밀하게 추적합니다:

```
// beacon_node/lighthouse_network/src/peer_manager/peerdb/peer_info.rs

#[derive(Debug, Clone, Default)]

pub  enum PeerConnectionStatus {

/// The peer is connected.

Connected {

multiaddr: Multiaddr,

n_in: u8, // 인바운드 연결 수

n_out: u8, // 아웃바운드 연결 수

},

/// The peer is being disconnected.
```

```
Disconnecting { to_ban: bool },

/// The peer has disconnected.

Disconnected { since: Instant },

/// The peer has been banned and is disconnected.

Banned { since: Instant },

/// We are currently dialing this peer.

Dialing { since: Instant },

/// The connection status has not been specified.

Unknown,

}
```

연결 방향 관리

```
#[derive(Debug, Clone, Copy, Serialize, AsRefStr)]

pub  enum ConnectionDirection {

/// The connection was established by a peer dialing us.

Incoming,

/// The connection was established by us dialing a peer.

Outgoing,

}
```

피어 평판 시스템

피어 액션과 스코어링

PeerManager는 피어의 행동에 따라 평판 점수를 조정합니다:

```
/// Reports a peer for some action.
pub fn report_peer(
&mut self,
peer_id: &PeerId,
action: PeerAction,
source: ReportSource,
reason: Option<GoodbyeReason>,
msg: &'static str,
) {
let action = self
.network_globals
.peers
.write()
.report_peer(peer_id, action, source, msg);
self.handle_score_action(peer_id, action, reason);
}
```

스코어 액션 처리

```
fn handle_score_action(
&mut self,
peer_id: &PeerId,
```

```
action: ScoreUpdateResult,
reason: Option<GoodbyeReason>,
) {
match  action {
ScoreUpdateResult::Ban(ban_operation) => {
self.handle_ban_operation(peer_id, ban_operation, reason);
}
ScoreUpdateResult::Disconnect => {
self.inbound_ping_peers.remove(peer_id);
self.outbound_ping_peers.remove(peer_id);
self.events.push(PeerManagerEvent::DisconnectPeer(
*peer_id,
GoodbyeReason::BadScore,
));
}
ScoreUpdateResult::NoAction => {
// The report had no effect on the peer
}
ScoreUpdateResult::Unbanned(unbanned_ips) => {
self.events
.push(PeerManagerEvent::UnBanned(*peer_id, unbanned_ips));
}
}
```

```
}
```

피어 수 관리 및 최적화

타겟 피어 수 관리

PeerManager는 다양한 피어 수 임계값을 관리합니다:

```
/// The maximum number of peers we allow to connect to us.
fn max_peers(&self) -> usize {
(self.target_peers as f32 * (1.0 + PEER_EXCESS_FACTOR)).ceil() as usize
}

/// The maximum number of peers we allow when dialing a priority peer.
fn max_priority_peers(&self) -> usize {
(self.target_peers as f32 * (1.0 + PEER_EXCESS_FACTOR + PRIORITY_PEER_EXCESS)).c
as usize
}

/// The minimum number of outbound peers before we start another discovery query
fn min_outbound_only_peers(&self) -> usize {
(self.target_peers as f32 * MIN_OUTBOUND_ONLY_FACTOR).ceil() as usize
}
```

피어 수 유지 로직

```
/// Queue another discovery if we need to
fn maintain_peer_count(&mut self, to_dial_peers: usize) {
let connected_or_dialing = self.network_globals.connected_or_dialing_peers();
// 아웃바운드 전용 피어가 부족한 경우 발견 쿼리 시작
if self.network_globals.connected_outbound_only_peers() < self.min_outbound_onl
&& connected_or_dialing + to_dial_peers < self.max_outbound_dialing_peers()
{
let peers_to_find = self.target_outbound_peers()
.saturating_sub(self.network_globals.connected_outbound_only_peers());
if peers_to_find > 0 {
self.events.push(PeerManagerEvent::DiscoverPeers(peers_to_find));
}
}
}
```

Discovery 서비스 구현

Discovery 구조체

```
// beacon_node/lighthouse_network/src/discovery/mod.rs
pub struct Discovery<E: EthSpec> {
/// A collection of seen live ENRs for quick lookup and to map peer-id's to ENRs
```

```
cached_enrs: LruCache<PeerId, Enr>,

/// The directory where the ENR is stored.

enr_dir: String,

/// The handle for the underlying discv5 Server.

discv5: Discv5,

/// A collection of network constants that can be read from other threads.

network_globals: Arc<NetworkGlobals<E>>,

/// Indicates if we are actively searching for peers.

find_peer_active: bool,

/// A queue of subnet queries to be processed.

queued_queries: VecDeque<SubnetQuery>,

/// Active discovery queries.

active_queries: FuturesUnordered<Pin<Box<dyn Future<Output = QueryResult> + Send

/// The discv5 event stream.

event_stream: EventStream,

/// Indicates if the discovery service has been started.

pub  started: bool,

/// Specifies whether various port numbers should be updated.

update_ports: UpdatePorts,

spec: Arc<ChainSpec>,

}
```

피어 발견 프로세스

일반 피어 발견

```
/// This adds a new `FindPeers` query to the queue if one doesn't already exist.
pub fn discover_peers(&mut self, target_peers: usize) {
if self.find_peer_active {
debug!("Discovery query already in progress, ignoring");
return;
}
let target_peers = std::cmp::min(FIND_NODE_QUERY_CLOSEST_PEERS, target_peers);
debug!(target_peers, "Starting a peer discovery request");
self.find_peer_active = true;
self.start_query(QueryType::FindPeers, target_peers, |_| true);
}
```

서브넷 피어 발견

```
/// Processes a request to search for more peers on a subnet.
pub fn discover_subnet_peers(&mut self, subnets_to_discover: Vec<SubnetDiscov
if !self.started {
return;
}
trace!(
subnets = ?subnets_to_discover.iter().map(|s| s.subnet).collect::<Vec<_>>(),
```

```
"Starting discovery query for subnets"
);
for subnet in subnets_to_discover {
self.add_subnet_query(subnet.subnet, subnet.min_ttl, 0);
}
}
```

서브넷 쿼리 관리

서브넷 쿼리 구조

```
#[derive(Clone, PartialEq)]
struct SubnetQuery {
subnet: Subnet,
min_ttl: Option<Instant>,
retries: usize,
}

impl SubnetQuery {
/// Returns true if this query has expired.
pub fn expired(&self) -> bool {
if let Some(ttl) = self.min_ttl {
ttl < Instant::now()
} else {
```

```
false // Long lived subnet discovery requests
}
}
}
```

서브넷 쿼리 처리

```
/// Runs a discovery request for a given group of subnets.
fn start_subnet_query(&mut self, subnet_queries: Vec<SubnetQuery>) {
let mut filtered_subnets: Vec<Subnet> = Vec::new();
let filtered_subnet_queries: Vec<SubnetQuery> = subnet_queries
.into_iter()
.filter(|subnet_query| {
// 충분한 피어가 있는지 확인
let peers_on_subnet = self
.network_globals
.peers
.read()
.good_peers_on_subnet(subnet_query.subnet)
.count();
if peers_on_subnet >= TARGET_SUBNET_PEERS {
debug!(
subnet = ?subnet_query.subnet,
connected_peers_on_subnet = peers_on_subnet,
```

```
target_subnet_peers = TARGET_SUBNET_PEERS,
"Discovery ignored"
);
return false;
}
true
})
.collect();
// 서브넷이 있는 경우에만 발견 쿼리 시작
if !filtered_subnet_queries.is_empty() {
// 서브넷 predicate를 eth2_fork_predicate와 결합하여 구축
let  subnet_predicate = subnet_predicate::<E>(filtered_subnets, &self.spec);
self.start_query(
QueryType::Subnet(filtered_subnet_queries),
TARGET_PEERS_FOR_GROUPED_QUERY,
subnet_predicate,
);
}
}
```

ENR (Ethereum Node Record) 관리

ENR 캐싱 및 관리

```
/// Add an ENR to the routing table of the discovery mechanism.
pub fn add_enr(&mut self, enr: Enr) {
// ENR을 캐시에 추가
let peer_id = enr.peer_id();
self.cached_enrs.put(peer_id, enr.clone());
// discv5 라우팅 테이블에 추가
if let Err(e) = self.discv5.add_enr(enr) {
debug!(
error = %e,
peer_id = %peer_id,
"Could not add peer to the local routing table"
);
}
}
```

ENR 업데이트

```
/// Updates the local ENR TCP port.
pub fn update_enr_tcp_port(&mut self, port: u16) -> Result<(), String> {
self.discv5
.enr_insert("tcp", &port.to_be_bytes())
.map_err(|e| format!("{:?}", e))?;
// 글로벌 버전을 discovery 버전으로 교체
```

```
*self.network_globals.local_enr.write() = self.discv5.local_enr();

Ok(())

}
```

PeerInfo 상세 구현

PeerInfo 구조체

```
// beacon_node/lighthouse_network/src/peer_manager/peerdb/peer_info.rs

#[derive(Clone, Debug, Serialize)]

pub  struct PeerInfo<E: EthSpec> {

/// The peers reputation

pub(crate) score: Score,

/// Client managing this peer

client: Client,

/// Connection status of this peer

connection_status: PeerConnectionStatus,

/// The known listening addresses of this peer

listening_addresses: Vec<Multiaddr>,

/// These are the multiaddrs we have physically seen

seen_multiaddrs: HashSet<Multiaddr>,

/// The current syncing state of the peer

sync_status: SyncStatus,
```

```
/// The ENR subnet bitfield of the peer
meta_data: Option<MetaData<E>>,
/// Subnets the peer is connected to
subnets: HashSet<Subnet>,
/// Subnets that the peer is assigned to custody
custody_subnets: HashSet<DataColumnSubnetId>,
/// The time we would like to retain this peer
min_ttl: Option<Instant>,
/// Is the peer a trusted peer
pub(crate) is_trusted: bool,
/// Direction of the first connection
connection_direction: Option<ConnectionDirection>,
/// The enr of the peer, if known
enr: Option<Enr>,
}
```

서브넷 관리

서브넷 구독 확인

```
/// Returns if the peer is subscribed to a given `Subnet` from the metadata.
pub fn on_subnet_metadata(&self, subnet: &Subnet) -> bool {
if let Some(meta_data) = &self.meta_data {
match subnet {
```

```
Subnet::Attestation(id) => {
return meta_data.attnets().get(**id as usize).unwrap_or(false)
}
Subnet::SyncCommittee(id) => {
return meta_data
.syncnets()
.is_ok_and(|s| s.get(**id as usize).unwrap_or(false))
}
Subnet::DataColumn(subnet_id) => {
return self.is_assigned_to_custody_subnet(subnet_id)
}
}
}
false
}
```

장기 서브넷 관리

```
/// Returns an iterator over the long-lived subnets if it has any.
pub fn long_lived_subnets(&self) -> Vec<Subnet> {
let mut long_lived_subnets = Vec::new();
// 메타데이터 확인
if let Some(meta_data) = self.meta_data.as_ref() {
```

```
for subnet in 0..=meta_data.attnets().highest_set_bit().unwrap_or(0) {
if meta_data.attnets().get(subnet).unwrap_or(false) {
long_lived_subnets.push(Subnet::Attestation((subnet as u64).into()));
}
}
if let Ok(syncnet) = meta_data.syncnets() {
for subnet in 0..=syncnet.highest_set_bit().unwrap_or(0) {
if syncnet.get(subnet).unwrap_or(false) {
long_lived_subnets.push(Subnet::SyncCommittee((subnet as u64).into()));
}
}
}
}
long_lived_subnets
}
```

연결 상태 관리

인바운드 연결 처리

```
/// Modifies the status to Connected and increases the number of ingoing connect
pub(super) fn connect_ingoing(&mut self, multiaddr: Multiaddr) {
self.seen_multiaddrs.insert(multiaddr.clone());
match &mut self.connection_status {
```

```
Connected { n_in, .. } => *n_in += 1,

Disconnected { .. }

| Banned { .. }

| Dialing { .. }

| Disconnecting { .. }

| Unknown => {

self.connection_status = Connected {

n_in: 1,

n_out: 0,

multiaddr,

};

self.connection_direction = Some(ConnectionDirection::Incoming);

}

}

}
```

아웃바운드 연결 처리

```
/// Modifies the status to Connected and increases the number of outgoing connec

pub(super) fn  connect_outgoing(&mut  self, multiaddr: Multiaddr) {

self.seen_multiaddrs.insert(multiaddr.clone());

match &mut  self.connection_status {

Connected { n_out, .. } => *n_out += 1,
```

```
Disconnected { .. }

| Banned { .. }

| Dialing { .. }

| Disconnecting { .. }

| Unknown => {

self.connection_status = Connected {

n_in: 0,

n_out: 1,

multiaddr,

};

self.connection_direction = Some(ConnectionDirection::Outgoing);

}

}

}
```

피어 밴 및 언밴 시스템

밴 작업 처리

```
/// If a peer is being banned, this handles the banning operation.

fn  handle_ban_operation(

&mut  self,

peer_id: &PeerId,
```

```
ban_operation: BanOperation,

reason: Option<GoodbyeReason>,

) {

match  ban_operation {

BanOperation::TemporaryBan => {

// 피어 타겟 한도에 도달한 경우에만 임시 밴

if  self.network_globals.connected_peers() >= self.target_peers {

self.temporary_banned_peers.raw_insert(*peer_id);

self.events.push(PeerManagerEvent::Banned(*peer_id, vec![]));

}

}

BanOperation::DisconnectThePeer => {

// 현재 연결된 피어를 연결 해제 시작

self.events.push(PeerManagerEvent::DisconnectPeer(

*peer_id,

reason.unwrap_or(GoodbyeReason::BadScore),

));

}

BanOperation::PeerDisconnecting => {

// 피어가 현재 연결 해제 중이며 완료 시 밴됨

}

BanOperation::ReadyToBan(banned_ips) => {

// 피어가 현재 연결되지 않음, swarm 레벨에서 안전하게 밴 가능
```

```
if !self.temporary_banned_peers.raw_remove(peer_id) {
self.events
.push(PeerManagerEvent::Banned(*peer_id, banned_ips));
}
}
}
}
```

피어 언밴

```
/// Unbans the peer in discovery.
pub fn unban_peer(&mut self, peer_id: &PeerId, ip_addresses: Vec<IpAddr>) {
// peer_id를 node_id로 변환 시도
if let Ok(node_id) = peer_id_to_node_id(peer_id) {
self.discv5.permit_node(&node_id);
}
// IP 주소들을 언밴
for ip_address in ip_addresses {
self.discv5.permit_ip(ip_address);
}
}
```

하트비트 및 유지보수

정기적 유지보수 작업

PeerManager는 정기적인 하트비트를 통해 다음 작업들을 수행합니다:

```
/// Performs the heartbeat maintenance on the peer manager.
pub fn heartbeat(&mut self) {
// 임시 밴된 피어들의 캐시 업데이트
self.temporary_banned_peers.update();
// 피어 수 유지
self.maintain_peer_count(0);
// 만료된 서브넷 정리
self.clean_expired_subnets();
// 피어 스코어 업데이트
self.update_peer_scores();
// 메트릭 업데이트
if self.metrics_enabled {
self.update_metrics();
}
}
```

서브넷 만료 관리

```
/// Clean expired sync committee subnets
fn clean_expired_subnets(&mut self) {
```

```
let now = Instant::now();
self.sync_committee_subnets.retain(|_, &mut min_ttl| min_ttl > now);
}
```

메트릭 및 모니터링

피어 관련 메트릭

PeerManager는 다양한 메트릭을 수집하여 네트워크 상태를 모니터링합니다:

```
fn update_metrics(&self) {
let connected_peers = self.network_globals.connected_peers();
let connected_outbound_only = self.network_globals.connected_outbound_only_peer
metrics::set_gauge(&metrics::PEERS_CONNECTED, connected_peers as i64);
metrics::set_gauge(&metrics::PEERS_OUTBOUND_ONLY, connected_outbound_only as i6
// 서브넷별 피어 수
for subnet in 0..self.network_globals.spec.attestation_subnet_count {
let subnet_peers = self.network_globals
.peers
.read()
.good_peers_on_subnet(Subnet::Attestation(subnet.into()))
.count();
metrics::set_gauge_vec(
&metrics::PEERS_PER_SUBNET,
```

```
&[&subnet.to_string()],
subnet_peers  as i64,
);
}
}
```

결론

Lighthouse의 피어 관리 및 디스커버리 시스템은 다음과 같은 핵심 기능들을 제공합니다:

1. **지능적 피어 관리**: 평판 기반 피어 스코어링과 자동 연결/해제
2. **효율적 피어 발견**: discv5 프로토콜을 통한 자동 피어 발견
3. **서브넷 최적화**: 검증자 듀티에 필요한 서브넷 피어 관리
4. **연결 상태 추적**: 세밀한 연결 상태 관리와 모니터링
5. **밴/언밴 시스템**: 악의적 피어에 대한 보호 메커니즘

이러한 시스템들이 협력하여 Lighthouse가 안정적이고 효율적인 P2P 네트워크를 유지할 수 있도록 합니다. 특히 서브넷 기반 피어 관리와 평판 시스템은 Ethereum 2.0의 대규모 네트워크에서 최적의 성능을 보장하는 핵심 요소입니다.